Contents

Chapter 26: User Testing **2**
From Working System to Validated Product . . . 2
1. The Product Is Now an Experimental System . . . 3
2. Why AI Products Require Special User Testing . . . 3
3. What You Are Trying to Discover . . . 4
1. Confusion . . . 4
2. Trust . . . 4
3. AI failure . . . 4
4. Latency . . . 4
5. Control . . . 4
6. Value . . . 4
4. Recruit the Right Users . . . 5
5. Test the Existing Workflow . . . 5
6. Give Users Tasks, Not Explanations . . . 6
7. The Think-Aloud Technique . . . 6
8. Observe Behavior, Not Just Opinions . . . 7
9. Where Users Get Confused . . . 7
10. Trust Is a First-Class Metric . . . 8
11. Trust Calibration . . . 8
12. Where the AI Fails . . . 9
13. Latency Is Part of the Product . . . 10
14. Streaming and Progressive Disclosure . . . 10
15. Users Want Control . . . 11
16. The Control Surface . . . 11
17. What Users Actually Value . . . 12
18. The “Magic Moment” . . . 12
19. The “Oh, That’s Useless” Moment . . . 13
20. Separate UX Problems From AI Problems . . . 13
21. User Testing Is an Instrumentation Problem . . . 14
22. Combine Qualitative and Quantitative Evidence . . . 15
23. Rank Problems by Severity . . . 15
24. Do Not Build Every Requested Feature . . . 16
25. Revise the Product Hypothesis . . . 17
26. The Product Development Loop . . . 17
27. Do Not Optimize Too Early . . . 17
28. The User-Testing Session . . . 18
Before the session . . . 18
During the session . . . 18
After the session . . . 19
29. Example Observation Log . . . 19
30. Chapter 26 Exercise . . . 19
31. The Deliverable . . . 21
1. Participants . . . 21
2. Tasks . . . 21

3. Observations 21
4. AI Failures 21
5. UX Problems 21
6. Trust Problems 21
7. Latency Problems 21
8. Control Requirements 21
9. Value Signals 21
10. Metrics 21
11. Prioritized Changes 22
12. Revised Product Hypothesis 22
13. Revised MVP 22
32. The Deeper Engineering Lesson 22
33. Why This Matters More for AI 22
34. The New Engineering Discipline 23
35. Key Takeaways 23

Chapter 26: User Testing

From Working System to Validated Product

Chapter 25 produced a working MVP.

Chapter 26 asks the more important question:

> **Does anyone actually want it?**

This distinction is fundamental.

A system can be:

- architecturally elegant,
- technically impressive,
- highly automated,
- well tested,
- powered by an excellent model,

and still be a bad product.

The only reliable way to discover this is to put the system in front of real users and observe what happens.

The central loop becomes:

$$\boxed{\text{Build} \rightarrow \text{Observe} \rightarrow \text{Learn} \rightarrow \text{Revise}}$$

This is fundamentally different from asking users:

> "Do you like the product?"

Users often cannot accurately predict what they will use.

Behavior is stronger evidence than opinion.

---

1. The Product Is Now an Experimental System

At this stage, stop thinking of the MVP primarily as software.

Treat it as an **experiment**.

You have a hypothesis:

$H =$ "This product provides meaningful value for this user performing this task."

User testing produces evidence:

$$E = e_1, e_2, ..., e_n$$

You then update your belief in (H).

Conceptually:

$$P(H|E)$$

The goal is not to prove that the original product is correct.

The goal is to discover where the hypothesis is wrong.

This changes the mindset from:

> "How do we convince users to like this?"

to:

> **"What can users teach us that we do not yet understand?"**

---

2. Why AI Products Require Special User Testing

Traditional software has predictable behavior.

If a user clicks a button, the system usually performs the same operation.

AI systems introduce another variable:

$$P(y|x)$$

The same input may produce different outputs.

Consequently, the user is evaluating both:

Product UX

and:

AI Behavior

A user might say:

"I don't trust this."

That could mean:

- the model is actually wrong,
- the system does not show evidence,
- the UI hides its reasoning process,
- confidence is poorly calibrated,
- the model sounds too certain,
- the user does not understand how the system works.

User testing helps distinguish these failure modes.

---

3. What You Are Trying to Discover

During testing, focus on six categories.

1. Confusion

Where does the user not understand what to do?

2. Trust

Where does the user doubt the system?

3. AI failure

Where does the system produce incorrect or misleading results?

4. Latency

Where does waiting interrupt the workflow?

5. Control

Where does the user want to intervene?

6. Value

Which capabilities actually matter?

These categories provide a practical observation framework:

$$\boxed{\text{Confusion} + \text{Trust} + \text{Failure} + \text{Latency} + \text{Control} + \text{Value}}$$

---

4. Recruit the Right Users

The quality of user testing depends heavily on participant selection.

Do not simply test with friends.

Ideally, users should resemble the target persona defined on Chapter 24.

For example, if the product is an AI incident investigator, test with:

- production engineers,
- SREs,
- on-call engineers,
- engineering managers who participate in incident response.

The important variable is not demographic similarity.

It is **workflow similarity**.

A user who regularly performs the target task can identify problems that a generic evaluator will miss.

---

5. Test the Existing Workflow

Do not immediately teach users how the new product works.

First understand how they currently solve the problem.

Ask:

> "Show me how you would normally do this."

Observe:

$$W_{\text{current}}$$

Then introduce the MVP and observe:

$$W_{\text{AI}}$$

The real comparison is:

$$\boxed{W_{\text{current}} \quad \text{vs.} \quad W_{\text{AI}}}$$

The product only creates value if:

$$U(W_{\text{AI}}) > U(W_{\text{current}})$$

where U represents the user's perceived or measured utility.

---

6. Give Users Tasks, Not Explanations

A common mistake is presenting a product and explaining every feature.

Instead, give the user a realistic task.

For example:

> "A production incident occurred. Investigate what happened and determine the likely root cause."

Then observe.

Do not immediately tell them:

- where to click,
- what the AI does,
- which buttons to use,
- what the expected answer is.

You want to observe whether the product communicates its own affordances.

This reveals:

- discoverability problems,
- confusing terminology,
- unnecessary UI elements,
- missing functionality,
- incorrect assumptions about user behavior.

---

7. The Think-Aloud Technique

A useful user-testing technique is **think-aloud testing**.

Ask users to verbalize what they are thinking:

> "Tell me what you're looking at and what you're trying to do."

You may hear:

> "I don't know what this means."
>
> "Why is it taking so long?"
>
> "I'm not sure whether this is reliable."
>
> "Where did this conclusion come from?"
>
> "I want to change the search parameters."

These statements expose the user's internal model of the system.

The key question is:

$$\text{User Mental Model} \stackrel{?}{=} \text{System Mental Model}$$

If they differ substantially, the interface or workflow needs improvement.

---

8. Observe Behavior, Not Just Opinions

User statements are useful, but behavior is often more informative.

A user may say:

> "This is really useful."

Then never use the feature again.

Another user may say:

> "I'm not sure about this."

but use it repeatedly because it saves substantial time.

Therefore collect:

$$\text{Declared Preference}$$

and:

$$\text{Observed Behavior}$$

Prefer the latter when they conflict.

Useful behavioral signals include:

- task completion,
- time to completion,
- number of errors,
- number of retries,
- abandoned tasks,
- features used,
- features ignored,
- manual work performed after AI output,
- frequency of overriding AI suggestions.

---

9. Where Users Get Confused

Confusion is valuable data.

Record every moment where users:

- stop,
- ask what something means,

- click the wrong control,
- repeat an action,
- look for missing information,
- misunderstand an AI result.

For each confusion point, record:

$$C_i = (\text{Context}, \text{User Action}, \text{Expected}, \text{Actual})$$

For example:

> Context: AI-generated root cause report User action: Searches for evidence supporting the conclusion Expected: Evidence should be visible Actual: Evidence is hidden behind another interaction

This converts vague feedback into an actionable product defect.

---

10. Trust Is a First-Class Metric

Trust is especially important for AI systems.

A user may accept ordinary software behavior automatically.

AI output requires a different decision:

$$\text{Should I believe this?}$$

Trust therefore depends on multiple factors:

$$T = f(A, E, C, P, X)$$

where:

- A = perceived accuracy,
- E = evidence,
- C = consistency,
- P = predictability,
- X = explainability/transparency.

A highly capable system can still fail if users cannot determine when it is reliable.

---

11. Trust Calibration

The objective is not maximum trust.

It is **appropriate trust**.

Two failure modes exist.

Under-trust The system is correct, but users ignore it.

$$T_{\text{user}} < T_{\text{appropriate}}$$

Over-trust

The system is unreliable, but users believe it.

$$T_{\text{user}} > T_{\text{appropriate}}$$

The second case is substantially more dangerous.

A good AI product should help the user distinguish:

- high-confidence conclusions,
- uncertain conclusions,
- unsupported claims,
- conflicting evidence.

The ideal state is:

$$\boxed{T_{\text{user}} \approx R_{\text{system}}}$$

where perceived reliability is aligned with actual reliability.

---

12. Where the AI Fails

User testing should intentionally expose failures.

Do not test only ideal scenarios.

Include:

- ambiguous inputs,
- incomplete information,
- contradictory evidence,
- unusual requests,
- irrelevant documents,
- missing tools,
- malformed data,
- long-context situations,
- adversarial or misleading inputs.

Observe:

$$x \to AI(x) \to \text{User Reaction}$$

The user may discover failures that the offline evaluation set missed.

This is why:

$$\boxed{\text{Offline Evaluation} \neq \text{Real-World Evaluation}}$$

Both are necessary.

---

13. Latency Is Part of the Product

Latency is not merely an infrastructure metric.

It is a UX variable.

Suppose an AI system takes:

$$t = 3s$$

for a simple operation.

That may feel instantaneous.

But:

$$t = 45s$$

may fundamentally change how the user interacts with the product.

They may:

- switch tasks,
- abandon the request,
- lose context,
- retry unnecessarily,
- stop using the feature.

For agentic systems, latency can be decomposed as:

$$T_{\text{total}} = T_{\text{LLM}} + T_{\text{retrieval}} + T_{\text{tools}} + T_{\text{orchestration}} + T_{\text{network}}$$

Measure both:

$$T_{\text{mean}}$$

and:

$$T_{p95}$$

or T_{p99} where appropriate.

Users often experience tail latency rather than average latency.

---

14. Streaming and Progressive Disclosure

Long-running AI operations do not necessarily need to feel slow.

The product can expose progress:

$$\text{Request} \rightarrow \text{Searching} \rightarrow \text{Analyzing} \rightarrow \text{Verifying} \rightarrow \text{Result}$$

This creates a perceived interaction model that is substantially better than:

$$\text{Request} \rightarrow \text{Blank Screen} \rightarrow \text{Result}$$

However, progress indicators must be truthful.

The system should not simulate progress merely to make the UI feel responsive.

---

15. Users Want Control

One of the most important discoveries in AI product design is that users frequently want **selective autonomy**.

They may want the AI to:

- gather information,
- summarize,
- recommend,
- draft,
- analyze.

But they may want to personally:

- approve actions,
- inspect evidence,
- modify parameters,
- correct mistakes,
- decide when to execute.

This produces a spectrum:

Manual → Assisted → Recommended → Supervised Autonomous → Autonomous

Do not assume that maximum autonomy is maximum value.

The optimal point depends on:

$$\text{Risk} + \text{Trust} + \text{Task Structure} + \text{User Preference}$$

---

16. The Control Surface

An AI product should provide users with appropriate control surfaces.

These might include:

- edit,
- regenerate,
- retry,
- stop,
- inspect evidence,
- modify search scope,
- approve,
- reject,
- override,
- undo.

The user should understand:

> "What is the AI doing?"

and:

> "What can I change?"

This becomes especially important when the AI takes multiple actions.

---

17. What Users Actually Value

This is perhaps the most important observation.

Users may tell you that they want:

- more features,
- more customization,
- more models,
- more automation.

But their actual behavior may reveal that one simple capability provides almost all the value.

Suppose your product has:

$$F = f_1, f_2, \dots, f_{20}$$

but users repeatedly rely on:

$$f_7$$

Then the product may actually be:

$$\text{Product} \approx f_7$$

rather than a 20-feature platform.

This is one of the reasons user testing should happen early.

---

18. The "Magic Moment"

Many AI products have a moment when the value becomes obvious.

For example:

> The system analyzes a problem that would normally require 30 minutes of manual work and produces a useful result in 60 seconds.

This is the **magic moment**.

Identify:

$$M_{\text{magic}}$$

and ask:

What happened immediately before the user recognized the value?

That moment may reveal the true product.

The MVP should increasingly optimize around the workflow that produces it.

---

19. The "Oh, That's Useless" Moment

The opposite discovery is equally valuable.

You may observe:

> User asks the AI to perform the task.
>
> AI produces an answer.
>
> User immediately opens another application and does the task manually.

That is important evidence.

Ask why.

Possible explanations:

- answer is inaccurate,
- evidence is insufficient,
- result is difficult to use,
- latency is too high,
- user does not trust the AI,
- existing workflow is easier.

Do not immediately patch the interface.

First identify the underlying cause.

---

20. Separate UX Problems From AI Problems

This distinction is critical.

Suppose a user says:

> "I can't tell why the AI thinks this is the root cause."

Possible problem:

$$P_{\text{UX}}$$

The evidence exists but is poorly presented.

Alternatively:

$$P_{\text{AI}}$$

The system genuinely cannot support the conclusion.

These require different fixes.

A useful diagnostic matrix is:

Problem	Likely Intervention
User cannot find feature	UX
User misunderstands output	UX / explanation
Output is factually wrong	Model / retrieval / tools
Output lacks evidence	Retrieval / architecture
Output takes too long	Architecture / model / UX
User wants approval before action	Workflow
User does not perceive value	Product
User distrusts correct results	Evidence / transparency / UX

Do not solve a product problem with a better model.

---

21. User Testing Is an Instrumentation Problem

You cannot learn from users if you cannot observe the system.

Instrument:

- sessions,
- requests,
- latency,
- model calls,
- tool calls,
- retrieval results,
- errors,
- user edits,
- overrides,
- retries,
- abandoned workflows.

A useful event model is:

$$e_i = (timestamp, user, action, context, result)$$

A session becomes:

$$S = (e_1, e_2, \dots, e_n)$$

This allows qualitative observations to be combined with quantitative evidence.

---

22. Combine Qualitative and Quantitative Evidence

Neither is sufficient alone.

Qualitative Provides:

- why users behave a certain way,
- what they think,
- what they distrust,
- what they find confusing.

Quantitative Provides:

- frequency,
- magnitude,
- latency,
- conversion,
- retention,
- error rates.

Together:

$$\boxed{\text{Product Insight} = \text{Qualitative Evidence} + \text{Quantitative Evidence}}$$

For example:

Qualitative:

> "Users don't trust the answer."

Quantitative:

> 62% of users inspect the source evidence before accepting it, and 31% manually verify the result elsewhere.

Now the problem becomes measurable.

---

23. Rank Problems by Severity

Do not treat every piece of feedback equally.

A useful prioritization function is:

$$Priority = Severity \times Frequency \times Value$$

You can further include implementation cost:

$$ROI = \frac{Severity \times Frequency \times Value}{Cost}$$

For example:

Problem	Severity	Frequency	Cost	Priority
Users cannot find evidence	High	High	Low	Very high
Report font too small	Low	Medium	Low	Low
Agent sometimes hallucinates	Critical	Low	High	High
20-second latency	High	High	Medium	Very high

This prevents the team from spending an entire day fixing cosmetic issues while the core workflow remains broken.

---

24. Do Not Build Every Requested Feature

Users will request features.

Some will be excellent.

Others will be distractions.

A request such as:

> "Can you add Slack integration?"

does not automatically imply:

> "We should build Slack integration."

Instead ask:

> **What underlying problem is the user trying to solve?**

The feature request:

$$F$$

may actually represent a desired outcome:

$$O$$

For example:

> "I want Slack integration"

might mean:

> "I need to share the result with my team."

The solution may not require a Slack integration at all.

---

25. Revise the Product Hypothesis

After user testing, return to the original hypothesis.

Suppose the original hypothesis was:

> "Users want an autonomous AI system that investigates incidents."

Testing may reveal:

> Users do not want autonomous investigation. They want a fast evidence-gathering assistant that they control.

That is not a failed test.

That is a successful learning event.

The hypothesis has been refined:

$$H_0 \to H_1$$

This is exactly what the MVP was designed to accomplish.

---

26. The Product Development Loop

The complete loop becomes:

Hypothesis → MVP → Users → Observation → Evidence → Revision → New Hypothesis

This is effectively an empirical optimization process.

You are searching over product designs:

$$P_1, P_2, \ldots, P_n$$

and attempting to maximize:

$$U(P)$$

where U represents user and business value.

Each user-testing cycle provides information about the shape of (U(P)).

---

27. Do Not Optimize Too Early

One of the biggest engineering traps is polishing before validation.

A team may spend weeks improving:

- prompt quality,
- model selection,

- infrastructure,
- UI design,
- caching,
- abstractions,
- performance.

Then discover:

> Users do not actually want the workflow.

This produces:

$$\text{Engineering Effort} \gg \text{Validated Product Value}$$

The better sequence is:

$$\text{Cheap Experiment} \rightarrow \text{Evidence} \rightarrow \text{Commit Resources}$$

not:

$$\text{Large Investment} \rightarrow \text{Hope} \rightarrow \text{Evidence}$$

This is the core principle of Chapter 26.

---

28. The User-Testing Session

A practical session can follow this structure.

Before the session

Define:

- target persona,
- task,
- success criteria,
- questions,
- metrics,
- hypotheses.

During the session

Observe:

1. How does the user start?
2. What do they expect?
3. Where do they hesitate?
4. What do they click?
5. What do they trust?
6. What do they verify?
7. Where does the AI fail?

8. What do they override?
9. What do they ignore?
10. What creates obvious value?

After the session

Record:

- observations,
- failures,
- quotes,
- metrics,
- hypotheses,
- proposed changes.

Avoid relying on memory.

---

29. Example Observation Log

A useful format is:

Time	Observation	Category	Severity	Hypothesis
02:15	User searches for source evidence	Trust	High	Evidence should be visible
04:32	User waits 18s and switches tabs	Latency	High	Workflow is too slow
07:11	User ignores autonomous suggestion	Control	Medium	User wants approval
10:04	User manually repeats AI search	AI failure	High	Retrieval is insufficient

This transforms user testing from informal conversation into engineering data.

---

30. Chapter 26 Exercise

Take the MVP built on Chapter 25.

Put it in front of **real target users**.

Ideally conduct several sessions rather than relying on a single participant.

For each user:

Step 1 — Establish the baseline Observe how they currently solve the problem.

Step 2 — Give them a realistic task Do not explain the solution unnecessarily.

Step 3 — Observe Record:

- confusion,
- hesitation,
- errors,
- trust,
- AI failures,
- latency,
- control requirements.

Step 4 — Measure Capture:

- task completion,
- time to completion,
- retries,
- overrides,
- feature usage,
- AI acceptance,
- manual fallback.

Step 5 — Interview Ask:

- What was useful?
- What was confusing?
- What did you distrust?
- What would you change?
- What would you use repeatedly?
- What would you pay for?
- What would prevent you from adopting it?

Step 6 — Rank findings Classify each issue as:

- product,
- UX,
- AI quality,
- retrieval,
- architecture,
- latency,
- trust,
- control.

Step 7 — Revise Select the highest-value changes.

Then repeat:

$$\boxed{\text{Test} \rightarrow \text{Revise} \rightarrow \text{Test Again}}$$

---

31. The Deliverable

By the end of Chapter 26, produce a **User Testing Report** containing:

1. Participants

Who tested the product and why they represent the target user.

2. Tasks

What users were asked to accomplish.

3. Observations

What actually happened.

4. AI Failures

Where the system produced incorrect, misleading, or low-value behavior.

5. UX Problems

Where users became confused or blocked.

6. Trust Problems

Where users questioned system reliability.

7. Latency Problems

Where waiting materially affected behavior.

8. Control Requirements

Where users wanted more or less autonomy.

9. Value Signals

What users repeatedly found useful.

10. Metrics

Quantitative measures of workflow performance.

11. Prioritized Changes

Ranked by expected impact.

12. Revised Product Hypothesis

State what you now believe about the product.

13. Revised MVP

Document what changes in the product as a result.

---

32. The Deeper Engineering Lesson

Chapter 26 teaches a principle that extends far beyond UX:

Software quality is not the same thing as product value.

You can optimize:

$$Q_{\text{software}}$$

while neglecting:

$$V_{\text{user}}$$

A successful AI product requires both:

$$\boxed{\text{Product Success} = f(\text{Technical Quality}, \text{User Value})}$$

Technical excellence without user value produces elegant irrelevance.

User value without sufficient technical quality produces an unreliable product.

The engineering objective is to find the intersection.

---

33. Why This Matters More for AI

AI makes software development dramatically cheaper.

That creates a paradox.

If implementation becomes cheap, then the cost of building unwanted software also falls.

Therefore the bottleneck moves.

Traditional constraint:

$$\text{Can we build it?}$$

AI-native constraint:

$$\boxed{\text{Should we build it?}}$$

And once the answer is yes:

$$\boxed{\text{What exactly should we build?}}$$

This makes product discovery increasingly important as AI coding capability improves.

---

34. The New Engineering Discipline

The emerging workflow is:

$$\text{Observe} \rightarrow \text{Hypothesize} \rightarrow \text{Specify} \rightarrow \text{Generate} \rightarrow \text{Evaluate} \rightarrow \text{Observe Again}$$

Notice that the loop does not end with deployment.

Deployment begins the next learning cycle.

This is particularly important for AI systems because model behavior, user behavior, and system behavior interact continuously.

The product is therefore not a static artifact.

It is an evolving socio-technical system:

$$\boxed{\text{Users} \leftrightarrow \text{AI} \leftrightarrow \text{Software} \leftrightarrow \text{Data}}$$

---

35. Key Takeaways

1. **Put the product in front of real users as early as possible.**
2. **Observe behavior rather than relying exclusively on what users say.**
3. **Test the existing workflow before testing the new product.** You need a baseline against which to measure improvement.
4. **AI products require testing of both UX and model behavior.**
5. **Trust is a first-class product requirement.** The goal is not maximum trust but appropriately calibrated trust.
6. **Latency is part of UX.** A technically acceptable response time may still be unacceptable within the user's workflow.
7. **Users often want selective autonomy rather than maximum autonomy.** Design explicit control surfaces for approval, inspection, correction, and override.

8. **Separate UX failures from AI failures.** Better models cannot fix every product problem.
9. **Combine qualitative and quantitative evidence.**

$$\boxed{\text{Product Insight} = \text{Observation} + \text{Measurement}}$$

10. **Rank problems instead of reacting to every piece of feedback.**
11. **Do not automatically build requested features.** Discover the underlying user need first.
12. **User testing is a hypothesis-testing mechanism.**

$$H_0 \rightarrow \text{MVP} \rightarrow \text{Users} \rightarrow E \rightarrow H_1$$

13. **The MVP should be revised based on evidence, not defended because engineering effort has already been invested.**
14. **The most important AI-engineering habit is empirical discipline:**

    Don't spend three weeks polishing something users don't want.

15. **As AI makes implementation cheaper, product discovery becomes more—not less—important.**

$$\boxed{\text{Cheaper Implementation} \Rightarrow \text{More Experiments} \Rightarrow \text{Faster Learning} \Rightarrow \text{Better Products}}$$

Chapter 26 therefore completes an important transition. Chapter 24 defined the product, Chapter 25 built the product, and Chapter 26 exposes the product to reality. **The real product-development loop begins when users interact with what you built.**